07 - Pandas basics

Part 1 of this course had no data in it at all. That was deliberate: we spent four weeks learning to write programs, and now we apply those programs at real tables.

This notebook is the first half of that job - **getting a file off the disk and into a shape you can work with**:

- NumPy arrays: the layer pandas is built on
- `Series` and `DataFrame`, and the index that ties them together
- Reading a data file, and writing one back out
- Meeting a dataset: shape, types, missing values
- Selecting columns and rows, and filtering with conditions
- Changing values
- Cleaning: missing data and types
- Making new columns, and why you should almost never do it with a loop
- Putting it together as a function

By the end we will have written a function that takes a file path and hands back a clean table.

> ⚠️ **Warning:** This notebook, and every notebook from here on, must run on the **`ban405`** kernel, the environment you built when we set up conda. Check the kernel selector at the top right before you run anything. The first code cell prints the pandas version, and it should say `3.0.3`. If it does not, you are on the wrong kernel, and the cells below may print something different from what the text describes.

In [ ]:
```
import numpy as np
import pandas as pd

print(pd.__version__)
```

NumPy arrays

pandas is built on top of a package called **NumPy**, and almost everything that surprises people about pandas is explained one level down, here. So we start with the layer underneath.

Here is a problem we've already encountered. A shop has four prices, and we want all of them with 25% VAT added.

Loading [MathJax]/extensions/Safe.js

In [ ]:
```
prices = [29.90, 35.00, 12.50, 22.70]

prices * 2
```

A list is a container of things, and multiplying a container repeats it.

With `1.25` it cannot even do that, because you cannot repeat a list one and a quarter times.

In [ ]:
```
# TypeError
prices * 1.25
```

In Part 1 the answer was a loop, or a list comprehension. NumPy gives us a third option: a container that **does** mean something arithmetic.

In [ ]:
```
prices_array = np.array([29.90, 35.00, 12.50, 22.70])

print(prices_array)
print(prices_array * 1.25)
```

Every element was multiplied, with no loop written anywhere. This is called **vectorization**, and it is the whole reason NumPy exists: the loop still happens, but it happens inside compiled code rather than in Python, which for large data is dramatically faster and much shorter to write.

Two arrays of the same length combine element by element, and `.sum()` adds an array up.

In [ ]:
```
quantities = np.array([3, 5, 0, 2])

revenue = prices_array * quantities

print(revenue)
print(revenue.sum())
```

An array has exactly one type

This is the fact that explains most of what comes later. A list can hold anything at all; an array holds one kind of thing, and it knows which kind.

In [ ]:
```
print(prices_array.dtype)
print(quantities.dtype)
```

If you build an array out of mixed values, NumPy does not keep the mixture - it converts everything to a type that can hold all of it.

In [ ]:
```
print(np.array([1, 2, 3.5]))
print(np.array([1, 2, "3"]))
```

Loading [MathJax]/extensions/Safe.js

The first became floats because a whole number can be written as a decimal without loss. The second became text, because that is the only type that can hold all three.

Missing values are floats

Real data has holes in it, and NumPy writes a hole as `np.nan` - "not a number". It matters that `nan` is itself a **decimal**.

In [ ]:
```
print(type(np.nan))

readings = np.array([12.5, 14.0, np.nan, 11.0])

print(readings)
print(readings.dtype)
```

Arrays answer questions

An array has methods that summarize it - `.mean()`, `.min()` and `.max()` among them. And comparing an array with a number gives you an array of `True` and `False`, one answer per element.

In [ ]:
```
temperatures = np.array([4.2, -3.1, 0.0, -8.5, 12.3])

print(temperatures.mean())
print(temperatures.min(), temperatures.max())
print(temperatures < 0)
```

That array of `True` and `False` is called a **boolean mask**. Note that summing it counts the `True` values, because `True` counts as 1 and `False` as 0.

In [ ]:
```
print((temperatures < 0).sum())
```

And putting a mask **inside the square brackets** keeps only the elements where it was `True`.

In [ ]:
```
print(temperatures[temperatures < 0])
```

Five comparisons, a count and a selection, with no `if` and no loop anywhere. This is also how pandas works.

> 💡 **Tip:** Arrays can have more than one dimension - `np.array([[1, 2], [3, 4]])` is a 2×2 grid, and `.shape` tells you the layout. We will not need that directly, but `.shape` turns up constantly on tables, where it means (rows, columns).

Loading [MathJax]/extensions/Safe.js

Your turn

Converting a whole list of temperatures used to take a loop. Do it without one.

```
celsius = [4.2, 18.5, 11.0, 22.3, 8.7, 16.1]
```

Turn that list into a NumPy array, convert the whole thing to Fahrenheit in a single expression using `F = 9 / 5 * C + 32`, and display the result together with its mean.

In [ ]:

```
```

Series: an array with labels

A pandas **Series** is a NumPy array with a second thing attached: an **index**, giving every value a label.

In [ ]:

```
emissions = pd.Series(
    [41.6, 434.3, 6.9, 12.4],
    index=["Norway", "Brazil", "Iceland", "Ghana"]
)

emissions
```

The values on the right are an ordinary NumPy array, and you can reach them with the `values` method.

In [ ]:

```
print(emissions.values)
print(emissions.values.dtype)
print(emissions.index)
```

Everything arrays could do, a Series can do - including the boolean mask. The difference is that the answers keep their labels.

In [ ]:

```
print(emissions.mean())
print()
print(emissions[emissions > 20])
```

The index is not decoration

When two Series are combined, pandas matches them up by their index labels, not by their position. Here are populations for the same four countries, listed in a different order and missing one of them.

In [ ]:

```
population = pd.Series(
    , 5.5, 0.4],
```

Loading [MathJax]/extensions/Safe.js

```
    index=["Brazil", "Norway", "Iceland"]
)

population
```

In [ ]:
```
emissions / population
```

Note that three separate things happened.

1. **Norway was divided by Norway**, even though Norway is first in one Series and second in the other. Position was ignored; the label was what mattered.
2. **Ghana came out as `NaN`.** It has emissions but no population here, so there is nothing to divide by. pandas did not raise an error and it did not silently drop Ghana - it kept the row and marked the answer as missing.
3. **The result came back in alphabetical order**, which is neither input's order. pandas built a new index out of both.

If you have ever wondered why a pandas calculation produced a column of `NaN` when both inputs looked fine, this is almost always the reason: the two things you combined did not have the same labels.

Your turn

Predict what each of these two expressions produces - the values **and** the labels - before you run anything. Then run them and check.

```
a = pd.Series([1, 2, 3], index=["x", "y", "z"])
b = pd.Series([10, 20, 30], index=["z", "y", "x"])

a + b

c = pd.Series([1, 2, 3], index=["x", "y", "z"])
d = pd.Series([10, 20], index=["x", "w"])

c + d
```

In [ ]:
```

```

DataFrame: columns that share an index

A **DataFrame** is a table: several Series side by side, all sharing one index. Building one from a dictionary shows the structure clearly - each key becomes a column name.

In [ ]:
```
sales = pd.DataFrame({
    "product":  ["coffee", "tea", "juice", "sandwich"],
    "price":    [29.90, 35.00, 12.50, 48.00],
```

Loading [MathJax]/extensions/Safe.js

```
    "quantity": [3, 5, 0, 2],
})

sales
```

In [ ]:
```
print(sales.shape)
print(sales.columns)
print(sales.index)
```

Note that since index here was not given, pandas supplied the positions 0, 1, 2, 3 - which is why an index often *looks* like row numbers even though it is not one.

Each column has its own type, and because a column is an array, each column has exactly **one** type. Applying `dtypes` shows all of them at once.

In [ ]:
```
sales.dtypes
```

A single column really is a Series, and its values really are a NumPy array.

In [ ]:
```
print(type(sales["price"]))
print(type(sales["price"].values))
```

Reading and writing data files

Typing a table out by hand is fine for four rows. Everything else arrives as a file.

The most common format by far is **CSV** - comma-separated values. A CSV is just plain text: the first line holds the column names, every line after it is one row, and the values are separated by commas. You can open one in a text editor and read it.

`pd.read_csv` turns one into a DataFrame, and `.head()` shows the first five rows - which is how you check that a file loaded the way you expected before doing anything else.

In [ ]:
```
titanic = pd.read_csv("../data/titanic.csv")

titanic.head()
```

Why `../data/`

Recall from the conda workshop that a notebook's **working directory** is the folder the notebook itself sits in. This notebook is in `week07`, and the data is not in `week07` - it is in a `data` folder next to it:

```
BAN405/
├── data/
│   └── titanic.csv
```

Loading [MathJax]/extensions/Safe.js

```
└── week07/
    └── 07-pandas-basics.ipynb     <- you are here
```

So `"titanic.csv"` would fail: there is no such file in `week07`. `".."` means *the folder above this one*, and from there `data/titanic.csv` is a real path.

If you get a `FileNotFoundError`, the file almost always exists and the path is almost always wrong.

Writing a file back out

`to_csv` writes a DataFrame back to disk, and it takes the same kind of path.

Where to write is a rule of its own in this course: **the `data` folder is only ever read from.** Anything a notebook produces goes in the notebook's own folder, so a file name with no folder in front of it is all the path you need.

Two arguments worth knowing:

- `sep` sets the separator - a comma by default, but you can ask for anything.
- `index=False` says *do not write the index out as a column*, which is almost always what you want: without it, pandas adds an unnamed extra column of row numbers, and whoever opens the file next gets a mystery column called `Unnamed: 0`. If you have ever received a spreadsheet with one of those in it, now you know where it came from.

In [ ]:
```
titanic.to_csv("titanic_pipe.csv", sep="|", index=False)
```

Same data, separated by `|` instead of by commas. Now read it back - first the obvious way.

In [ ]:
```
pd.read_csv("titanic_pipe.csv").head(3)
```

One column, whose *name* is the entire header line. That is the signature of a wrong separator, and it is worth recognizing on sight, because you will meet files where nobody told you what the separator was. Here we know, because we wrote it.

In [ ]:
```
pd.read_csv("titanic_pipe.csv", sep="|").head(3)
```

Excel files

Spreadsheets work the same way, with `to_excel` and `read_excel` instead.

In [ ]:
```
titanic.to_excel("titanic.xlsx", index=False)
```

```
cel("titanic.xlsx").head(3)
```

Loading [MathJax]/extensions/Safe.js

Note that `titanic_pipe.csv` and `titanic.xlsx` are **derived** files: they contain nothing that is not in `titanic.csv`, and you could delete both and regenerate them from the code above. `titanic.csv` is the **source**, which is why the two derived files went into this week's folder rather than into `data`. Keeping that line clear - what came from outside, and what you made - is essential for making an analysis reproducible.

> ⚠️ **Warning:** pandas will hapily overwrite files on disk without asking. There is no "are you sure"...

Your turn

Starting from:

```
titanic = pd.read_csv("../data/titanic.csv")
```

Write the first 20 rows to a **semicolon**-separated file called `sample.csv`, read it back into a new variable, and print its shape. (`.head(20)` does not only display rows: it returns them as a DataFrame, which you can write out.)

The shape should be (20, 7). If you get (20, 8), find out where the extra column came from.

In [ ]:

Meeting a dataset

We now have a table in memory. Before doing anything with it, we need to find out what it is.

This one is the passenger list of the *Titanic*, which sank in 1912. Each row is one passenger:

Column	
`PassengerId`	a number identifying the passenger
`Survived`	1 if they survived, 0 if not
`Pclass`	ticket class: 1st, 2nd or 3rd
`Name`	last name, first name
`Sex`	"male" or "female"
`Age`	in years
`Fare`	what they paid, in pounds

Loading [MathJax]/extensions/Safe.js

`shape` gives you the number of rows and columns.

In [ ]:
```
print(titanic.shape)

titanic.head()
```

`info()` answers three questions at once: what the columns are called, how many values each one actually has, and what type each one is.

In [ ]:
```
titanic.info()
```

The table shows that every column has 891 values except `Age`, which has 714. So 177 ages are missing.

Notice the type of `Age` in the line beside it: `float64`.

Almost every age in this dataset is a whole number, so why is the column a decimal? Because 177 of the values are missing, missing is `NaN`, and **`NaN` is a float**.

`describe()` summarizes every numeric column at once.

In [ ]:
```
titanic.describe()
```

Note that it happily describes `Survived` and `Pclass` as though they were quantities. The mean of `Survived` is genuinely useful - it is the survival rate, 38% - while the mean of `Pclass` is arithmetic on labels and means nothing at all.

It also hands back a DataFrame, so you can pull single numbers straight out of it later.

Selecting columns and rows

For the rest of this notebook we move to a dataset we will keep coming back to for the whole second half of the course. It holds CO_2 emissions and a handful of economic indicators from the World Bank, one row per country per year, from 2000 to 2023.

In [ ]:
```
co2 = pd.read_csv("../data/co2_emissions.csv")

co2.head(3)
```

Square brackets on a DataFrame select **columns**. One name gives a Series; a list of names gives a smaller DataFrame.

In [ ]:
```
co2["co2_total"].head(3)
```

In [ ]:
```
co2[["country", "year", "co2_total"]].head(3)
```

Loading [MathJax]/extensions/Safe.js

Because `[]` means columns, asking for a number does not give you a row.

In [ ]:
```
# KeyError
co2[3]
```

pandas looked for a column called `3`, did not find one, and said so. Rows come from `.loc` and `.iloc`, which take rows first and columns second.

`.loc` speaks labels.

In [ ]:
```
co2.loc[0:3, ["country", "year", "co2_total"]]
```

`.iloc` speaks positions.

In [ ]:
```
co2.iloc[0:3, 0:3]
```

Look at how many rows each returned. `.loc[0:3]` gave **four** rows and `.iloc[0:3]` gave **three**, from the same table.

That is not an inconsistency to memorize. It follows from what each one means:

- `.loc` is asking for *the row labeled 3*, and you cannot include a label "up to but not including" it - either you want that row or you do not. So `.loc` is **inclusive** at both ends.
- `.iloc` is asking for *position 3*, and positions in Python have always stopped before the end value - the same rule as `sentence[0:4]` and `range(5)` from Part 1. So `.iloc` is **exclusive**.

The index here happens to be numbers, which is what makes the two look confusable. They are not: one is a name that happens to look like a number, the other is a count.

> 📝 **Note:** `.loc` and `.iloc` only look interchangeable here because pandas numbered the rows 0, 1, 2,... when it read the file, so every label happens to equal its position. With any other labels the difference is obvious.

Looking at one column

Once you can pull out a single column, a second set of questions becomes available - the ones about what is *in* a column rather than about the table as a whole. `.nunique()` counts how many distinct values it holds, and `.value_counts()` counts how often each one appears.

In [61]:
```
print(co2["country"].nunique(), "entities")
'year"].min(), "to", co2["year"].max())
```

Loading [MathJax]/extensions/Safe.js

```
print()
print(co2["year"].value_counts().head(3))
```

```
260 entities
2000 to 2023

year
2000    260
2001    260
2002    260
Name: count, dtype: int64
```

260 entities, 24 years, and every year has exactly 260 rows. That is a **balanced panel** - unusually tidy, and worth knowing, because it means a missing value is a hole in the data rather than a row nobody collected.

Applying `.unique()` on a column gives you a list-like object of the *unique* values in a the column.

In [ ]:
```
co2["country"].unique()
```

Before performing any operation on data, you should always check if the data contains missing values.

`isna()` gives one `True` or `False` per value - a boolean mask, exactly like the ones we built on arrays - and summing it counts the `True` values.

In [ ]:
```
print(co2["renew_energy"].isna().sum())
```

Apply it on the whole table at once and you get a column-by-column answer.

In [ ]:
```
co2.isna().sum()
```

Two indicators are complete and the rest are not. `nat_resources` is missing 759 times out of 6 240. Whether that matters depends entirely on what you are about to do with it.

Your turn

Starting from:

```
titanic = pd.read_csv("../data/titanic.csv")
```

Write one line of code for each of these three questions:

- How many passengers traveled in each class?
- How many passengers have no recorded age?
- What are the names and ages of the passengers in rows 10 to 14? Use `.loc`.

Loading [MathJax]/extensions/Safe.js

In [59]:
```
titanic.head(3)
```

Out[59]:

	PassengerId	Survived	Pclass	Name	Sex	Age	Fare
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	7.2500
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	71.2833
2	3	1	3	Heikkinen, Miss Laina	female	26.0	7.9250

In [ ]:
```
titanic["Pclass"].value_counts() # How many passengers traveled in each clas
```

```
Pclass
3    491
1    216
2    184
Name: count, dtype: int64
```

In [ ]:
```
print(titanic["Age"].isna().sum()) # How many passengers have no recorded ag
```

```
177
```

In [68]:
```
# What are the names and ages of the passengers from row 10 to 14? use .loc
titanic.loc[10:14, ["Name", "Age"]]
```

Out[68]:

	Name	Age
10	Sandstrom, Miss Marguerite Rut	4.0
11	Bonnell, Miss Elizabeth	58.0
12	Saundercock, Mr. William Henry	20.0
13	Andersson, Mr. Anders Johan	39.0
14	Vestrom, Miss Hulda Amanda Adolfina	14.0

Filtering rows with a condition

Comparing a column with a value gives one `True` or `False` per row. This is the third time we have built a boolean mask - first on an array, then on a Series, now on a column of a table - and it is the same idea every time.

In [69]:
```
mask = co2["year"] == 2023

print(type(mask))
print(mask.head(3))
print("rows that are True:", mask.sum())
```

Loading [MathJax]/extensions/Safe.js

```
<class 'pandas.Series'>
0    False
1    False
2    False
Name: year, dtype: bool
rows that are True: 260
```

Putting a mask in square brackets keeps the rows where it is `True`, exactly as it kept the elements of an array.

In [70]:
```
co2[co2["year"] == 2023].head()
```

Out[70]:

	country	code	year	co2_total	population	urban	gdp_pc	electricity	agri
23	Afghanistan	AFG	2023	11.6486	41454761	25.473	413.76	85.30	
47	Africa Eastern and Southern	AFE	2023	619.3657	750491370	37.772	1571.13	50.67	
71	Africa Western and Central	AFW	2023	251.6472	508318102	52.302	1846.25	57.06	
95	Albania	ALB	2023	4.2226	2411658	58.211	9740.70	100.00	
119	Algeria	DZA	2023	190.5786	46164219	74.733	5370.48	100.00	

Combining conditions

Two conditions are joined with `&` for "and" and `|` for "or", and `~` negates one.

Each condition must be wrapped in its own brackets, and that is not a style preference.

In [71]:
```
rich_2023 = co2[(co2["year"] == 2023) & (co2["gdp_pc"] > 50_000)]

print(len(rich_2023), "rows")
rich_2023[["country", "year", "gdp_pc"]].head()
```

```
31 rows
```

Out[71]:

	country	year	gdp_pc
335	Australia	2023	65058.38
359	Austria	2023	56579.50
527	Belgium	2023	55244.68
599	Bermuda	2023	132961.76
935	Canada	2023	54847.54

Loading [MathJax]/extensions/Safe.js

The brackets are needed because `&` binds **tighter** than `==` does. Without them, Python groups the expression the wrong way round, and pandas raises an error instead of returning any rows. Recall that this is the operator precedence we met when building conditions in Part 1.

And the English words do not work here at all.

In [ ]:
```
# ValueError
co2[(co2["year"] == 2023) and (co2["country"] == "Norway")]
```

`and` needs to know whether the whole left-hand side is true or false, and a mask of 6 240 answers is not one answer. `&` is the version that works element by element - which is what we actually want.

Several values: `.isin()`

To keep the rows for three countries, `|` works - but it takes one comparison per country, each in its own brackets, and one more pair around all of them so that `&` applies to the whole group.

In [72]:
```
nordic = co2[
    (co2["year"] == 2023)
    & ((co2["country"] == "Norway") | (co2["country"] == "Sweden") | (co2["c
]

nordic[["country", "year", "co2_total", "gdp_pc"]]
```

Out[72]:

	country	year	co2_total	gdp_pc
1415	Denmark	2023	26.2104	68043.55
4295	Norway	2023	41.6042	90984.41
5519	Sweden	2023	36.6683	54950.28

`.isin()` asks whether each value is one of a list, so the whole `|` chain becomes one condition.

In [73]:
```
nordic = co2[(co2["year"] == 2023) & (co2["country"].isin(["Norway", "Sweder

nordic[["country", "year", "co2_total", "gdp_pc"]]
```

Out[73]:

	country	year	co2_total	gdp_pc
1415	Denmark	2023	26.2104	68043.55
4295	Norway	2023	41.6042	90984.41
5519	Sweden	2023	36.6683	54950.28

Loading [MathJax]/extensions/Safe.js

And `~` turns a mask around: `~co2["country"].isin([...])` keeps every row *except* those countries.

In [74]:
```
others = co2[~co2["country"].isin(["Norway", "Sweden", "Denmark"])]

print(len(co2), "rows in all,", len(others), "without the three")
```

```
6240 rows in all, 6168 without the three
```

> ## Your turn
>
> Starting from:
>
> ```
> co2 = pd.read_csv("../data/co2_emissions.csv")
> ```
>
> Find the Nordic countries - Denmark, Finland, Iceland, Norway and Sweden - where renewable energy was above 50% of final consumption in 2020. Display each country with its renewable share.

In [75]:
```
co2.head(1)
```

Out[75]:

	country	code	year	co2_total	population	urban	gdp_pc	electricity	agricul
0	Afghanistan	AFG	2000	1.0093	20130327	18.558	174.93	4.4	57

In [ ]:
```
result = co2[
    (co2["year"]==2020)
    & (co2["country"].isin(["Denmark", "Finland", "Iceland", "Norway", "Swec
    & (co2["renew_energy"]>50)
    ]

result.head()
```

Out[ ]:

	country	code	year	co2_total	population	urban	gdp_pc	electricity	agric
2612	Iceland	ISL	2020	3.0717	366463	94.137	60127.94	100.0	
4292	Norway	NOR	2020	43.7079	5379475	82.703	71057.59	100.0	
5516	Sweden	SWE	2020	38.0388	10353442	87.942	52568.57	100.0	

Changing values

Suppose we want to correct a value. The obvious thing to write does not work.

In [ ]:
```
# This produces a warning and changes nothing.
co2[co2["country"] == "Norway"]["co2_total"] = 0

2["country"] == "Norway", "co2_total"].head(3)
```

Loading [MathJax]/extensions/Safe.js

Norway is untouched. `co2[co2["country"] == "Norway"]` built a **new table** containing the Norwegian rows, and the assignment changed that new table, which was then thrown away. pandas spots the pattern and warns you, because it is never what anybody means.

This is called **chained assignment** - two square-bracket operations in a row, with an `=` at the end.

The fix is to do the whole thing in one step, with `.loc`: rows first, then the column.

In [ ]:
```
co2.loc[co2["country"] == "Norway", "co2_total"] = 0
```

In [ ]:
```
co2.loc[co2["country"] == "Norway", "co2_total"].head(3)
```

> 📝 **Note:** A subset really is a separate table. `subset = co2[co2["year"] == 2023]` and then

changing `subset` leaves `co2` alone. In current pandas that is the rule for everything: anything you take out of a table - a subset, a single column, the result of a method - behaves as its own copy. The only way to change `co2` is to assign to `co2` itself, with `.loc`.

Having deliberately damaged the table, we read it again from disk.

In [ ]:
```
co2 = pd.read_csv("../data/co2_emissions.csv")
```

Cleaning

Everything from here changes the table. Before any of it, one rule that covers all of it: **A pandas method does not change your table. It returns a new one.**

So you either assign the result back to a name, or you pass it straight into the next method. This is the opposite of the list methods from Part 1, where `prices.sort()` rearranged the list in place and returned nothing.

> 📝 **Note:** You will see `inplace=True` in older code and in answers online. It still works, but pandas now

recommends against it, and everything in this course assigns instead.

Missing values: first look, then decide

We already know how much is missing. The more useful question is **where**.

Loading [MathJax]/extensions/Safe.js

In [84]:
```
for year in [2018, 2019, 2020, 2021, 2022, 2023]:
    rows = co2[co2["year"] == year]
    print(year, rows["renew_energy"].isna().sum())
```

```
2018 5
2019 5
2020 5
2021 48
2022 189
2023 260
```

That is not random at all. Five entities are missing in 2018, 2019 and 2020 - and then 48, 189, and finally **all 260** in 2023. The renewable energy series is simply published with a lag, so the most recent years are not there yet.

Dropping every row with a hole in it would silently throw away the three most recent years for every country in the dataset. `dropna()` is the method that does the throwing away.

`dropna()` with no arguments removes any row with a hole **anywhere**. Here that is 1 090 rows - about one row in six - thrown away mostly because of a column you may not even be using.

In [85]:
```
print("all rows:                ", len(co2))
print("after dropna():          ", len(co2.dropna()))
```

```
all rows:                 6240
after dropna():           5150
```

`subset=` restricts it to the columns you actually need, and costs 336. Almost always, `subset=` is the one you want.

In [86]:
```
print("after dropna on co2 only:", len(co2.dropna(subset=["co2_total"])))
```

```
after dropna on co2 only: 5904
```

The alternative is to fill the holes with `fillna()`, and that is a decision rather than a repair.

In [87]:
```
filled = co2.fillna({"renew_energy": co2["renew_energy"].mean()})

print("missing before:", co2["renew_energy"].isna().sum())
print("missing after: ", filled["renew_energy"].isna().sum())
```

```
missing before: 635
missing after:  0
```

> ⚠️ **Warning:** Filling with the mean is not free. Every filled row now sits exactly at the

Loading [MathJax]/extensions/Safe.js

average, so the spread of the data shrinks and any relationship you measure gets weaker. Here it would be worse than that: the missing values are concentrated in recent years, so filling them with a 24-year average invents a flat trend that never happened. Whether to drop, fill, or leave a hole alone is part of the analysis, not a tidying-up step before it.

Types

`astype` converts a column to another type. Like every method, it returns a new column rather than changing the old one, so the result has to be assigned back. `Survived` holds 0 and 1, which really mean `False` and `True`:

In [89]:
```
titanic["Survived"] = titanic["Survived"].astype(bool) #"translates" 1 and 0

titanic.head(3)
```

Out[89]:

	PassengerId	Survived	Pclass	Name	Sex	Age	Fare
0	1	False	3	Braund, Mr. Owen Harris	male	22.0	7.2500
1	2	True	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	71.2833
2	3	True	3	Heikkinen, Miss Laina	female	26.0	7.9250

`astype` also refuses to do the impossible, which is good.

In [90]:
```
# IntCastingNaNError
titanic["Age"].astype(int)
```

Loading [MathJax]/extensions/Safe.js

```
---------------------------------------------------------------------------
IntCastingNaNError                        Traceback (most recent call last)
Cell In[90], line 2
      1 # IntCastingNaNError
----> 2 titanic["Age"].astype(int)

File ~/miniforge3/envs/ban405/lib/python3.14/site-packages/pandas/core/generic.py:6541, in NDFrame.astype(self, dtype, copy, errors)
   6537             results = [ser.astype(dtype, errors=errors) for _, ser in self.items()]
   6538
   6539         else:
   6540             # else, only a single dtype is given
-> 6541             new_data = self._mgr.astype(dtype=dtype, errors=errors)
   6542             res = self._constructor_from_mgr(new_data, axes=new_data.axes)
   6543             return res.__finalize__(self, method="astype")
   6544

File ~/miniforge3/envs/ban405/lib/python3.14/site-packages/pandas/core/internals/managers.py:614, in BaseBlockManager.astype(self, dtype, errors)
    613 def astype(self, dtype, errors: str = "raise") -> Self:
--> 614     return self.apply("astype", dtype=dtype, errors=errors)

File ~/miniforge3/envs/ban405/lib/python3.14/site-packages/pandas/core/internals/managers.py:445, in BaseBlockManager.apply(self, f, align_keys, **kwargs)
    443         applied = b.apply(f, **kwargs)
    444     else:
--> 445         applied = getattr(b, f)(**kwargs)
    446     result_blocks = extend_blocks(applied, result_blocks)
    448 out = type(self).from_blocks(result_blocks, [ax.view() for ax in self.axes])

File ~/miniforge3/envs/ban405/lib/python3.14/site-packages/pandas/core/internals/blocks.py:607, in Block.astype(self, dtype, errors, squeeze)
    604         raise ValueError("Can not squeeze with more than one column.")
    605     values = values[0, :]  # type: ignore[call-overload]
--> 607 new_values = astype_array_safe(values, dtype, errors=errors)
    609 new_values = maybe_coerce_values(new_values)
    611 refs = None

File ~/miniforge3/envs/ban405/lib/python3.14/site-packages/pandas/core/dtypes/astype.py:240, in astype_array_safe(values, dtype, copy, errors)
    237     dtype = dtype.numpy_dtype
    239 try:
--> 240     new_values = astype_array(values, dtype, copy=copy)
    241 except (ValueError, TypeError):
    242     # e.g. _astype_nansafe can fail on object-dtype of strings
    243     #  trying to convert to float
    244     if errors == "ignore":

File ~/miniforge3/envs/ban405/lib/python3.14/site-packages/pandas/core/dtypes/astype.py:185, in astype_array(values, dtype, copy)
    182     values = values.astype(dtype, copy=copy)
```

Loading [MathJax]/extensions/Safe.js

```
    184 else:
--> 185     values = _astype_nansafe(values, dtype, copy=copy)
    187 # in pandas we don't store numpy str dtypes, so convert to object
    188 if isinstance(dtype, np.dtype) and issubclass(values.dtype.type, st
r):

File ~/miniforge3/envs/ban405/lib/python3.14/site-packages/pandas/core/dtype
s/astype.py:102, in _astype_nansafe(arr, dtype, copy, skipna)
     97     return lib.ensure_string_array(
     98         arr, skipna=skipna, convert_na_value=False
     99     ).reshape(shape)
    101 elif np.issubdtype(arr.dtype, np.floating) and dtype.kind in "iu":
--> 102     return _astype_float_to_int_nansafe(arr, dtype, copy)
    104 elif arr.dtype == object:
    105     # if we have a datetime/timedelta array of objects
    106     # then coerce to datetime64[ns] and use DatetimeArray.astype
    108     if lib.is_np_dtype(dtype, "M"):

File ~/miniforge3/envs/ban405/lib/python3.14/site-packages/pandas/core/dtype
s/astype.py:146, in _astype_float_to_int_nansafe(values, dtype, copy)
    142 """
    143 astype with a check preventing converting NaN to a meaningless integ
er value.
    144 """
    145 if not np.isfinite(values).all():
--> 146     raise IntCastingNaNError(
    147         "Cannot convert non-finite values (NA or inf) to integer."
    148         "Replace or remove non-finite values or cast to an integer t
ype"
    149         "that supports these values (e.g. 'Int64')"
    150     )
    151 if dtype.kind == "u":
    152     # GH#45151
    153     if not (values >= 0).all():

IntCastingNaNError: Cannot convert non-finite values (NA or inf) to integer.
Replace or remove non-finite values or cast to an integer typethat supports
these values (e.g. 'Int64')
```

There is no whole number that means "missing", so pandas will not guess. Either fill the holes first or drop those rows - the error is asking you to make the decision it cannot make for you.

So drop them, and assign the converted column back:

In [92]:
```
known_age = titanic.dropna(subset=["Age"])
known_age["Age"] = known_age["Age"].astype(int)

print(len(titanic), "->", len(known_age), "passengers")
print(known_age["Age"].dtype)
```

```
891 -> 714 passengers
int64
```

Loading [MathJax]/extensions/Safe.js

That worked. It also did something it did not mention. A few ages are not whole numbers at all - babies are recorded in fractions of a year, and estimated ages end in .5 - and `astype(int)` does not round them. It cuts the decimals off.

In [91]:
```
ages = titanic["Age"].dropna()
fractional = ages[ages != ages.round()]

print(len(fractional), "ages are not whole numbers")
print(fractional.head(4).tolist())
print(fractional.head(4).astype(int).tolist())
```

```
25 ages are not whole numbers
[28.5, 0.83, 14.5, 70.5]
[28, 0, 14, 70]
```

> ⚠ **Warning:** `astype(int)` truncates without a warning: 0.83 becomes 0 and 70.5 becomes 70.

If the decimals matter, `.round()` first and convert second.

When text needs to become numbers and some of it is not numeric, `pd.to_numeric` with `errors="coerce"` converts what it can and turns the rest into `NaN` rather than raising. This is the `try` / `except` idea from Part 1, built into pandas.

In [ ]:
```
messy = pd.Series(["12.5", "14.0", "n/a", "", "11.0"])

print(pd.to_numeric(messy, errors="coerce")) # converts what it can convert,
```

```
0    12.5
1    14.0
2     NaN
3     NaN
4    11.0
dtype: float64
```

Chaining

Because every method returns a new table, several of them can be strung together in one expression, read top to bottom. The one new method below is `sort_values`, which does what it says.

In [94]:
```
complete = (
    co2
    .dropna(subset=["co2_total", "gdp_pc"])
    .sort_values(["country", "year"])
)

print(complete.shape)
complete.head()
```

Loading [MathJax]/extensions/Safe.js

Out[94]:

	country	code	year	co2_total	population	urban	gdp_pc	electricity	agricul
0	Afghanistan	AFG	2000	1.0093	20130327	18.558	174.93	4.4	57
1	Afghanistan	AFG	2001	0.9402	20284307	18.741	138.71	9.3	57
2	Afghanistan	AFG	2002	0.9388	21378117	18.941	178.95	14.1	57
3	Afghanistan	AFG	2003	1.0172	22733049	19.160	198.87	19.0	58
4	Afghanistan	AFG	2004	0.9024	23560654	19.399	221.76	23.8	58

The brackets around the whole expression are what let it span several lines. Use chaining when the steps read as one thought, and break it up when they do not.

Your turn

Starting from:

```
co2 = pd.read_csv("../data/co2_emissions.csv")
```

You are about to study GDP per capita against CO_2 emissions in 2023. Build that table:

- keep only the year 2023;
- keep only the rows where **both** `gdp_pc` and `co2_total` are present;
- store GDP per capita as whole dollars;
- sort it from richest to poorest (`sort_values` has an `ascending` parameter).

Report how many rows you kept and how many you lost, and display the top five.

In [ ]:

```
result = (
    co2[co2["year"] == 2023]
    .dropna(subset=["gdp_pc", "co2_total"])
    .sort_values(["gdp_pc"], ascending=False)
    )

# we have to do the rounding outside the chain, because this is done on a co
# and not on the dataframe like the other functions
result["gdp_pc"] = result["gdp_pc"].round().astype(int)

print(result.head(5))
```

Loading [MathJax]/extensions/Safe.js

```
             country code  year  co2_total  population    urban  gdp_pc  \
3431      Luxembourg  LUX  2023     6.9416      666430   94.802  133231
599          Bermuda  BMU  2023     0.2583       64698  100.000  132962
2735         Ireland  IRL  2023    32.8629     5311538   64.115  106819
5543     Switzerland  CHE  2023    33.6661     8888822   85.185  104450
983   Cayman Islands  CYM  2023     0.3594       73038  100.000  100065

      electricity  agriculture  nat_resources  renew_energy
3431        100.0       51.661            NaN           NaN
599         100.0        5.556            NaN           NaN
2735        100.0       60.807            NaN           NaN
5543        100.0       37.794            NaN           NaN
983         100.0       11.250            NaN           NaN
```

Making a new column

A new column is an assignment, and the right-hand side is an ordinary calculation on the columns you already have. No loop: this is the vectorization from the very first section of this notebook, now working on a table.

In [103…

```
co2["co2_pc"] = co2["co2_total"] * 1_000_000 / co2["population"]

co2[["country", "year", "co2_total", "population", "co2_pc"]].head(3)
```

Out[103…

	country	year	co2_total	population	co2_pc
0	Afghanistan	2000	1.0093	20130327	0.050138
1	Afghanistan	2001	0.9402	20284307	0.046351
2	Afghanistan	2002	0.9388	21378117	0.043914

6 240 divisions, one line, no loop.

That was easy because the rule was arithmetic. The interesting case is a column whose value depends on a **condition**.

The instinct from Part 1

Suppose we want to label each country by income group. In Part 1 you would write a function and call it on each value, and pandas will happily let you do exactly that with `.apply()`.

In [104…

```
def classify_income(gdp_per_capita):
    """Return the World Bank income group for a GDP per capita figure."""
    if gdp_per_capita < 1_000:
        return "low"
    elif gdp_per_capita < 10_000:
        return "lower-middle"
    elif gdp_per_capita < 50_000:
        return "upper-middle"
```

Loading [MathJax]/extensions/Safe.js

```
    else:
        return "high"
```

In [ ]:
```
co2["income_group"] = co2["gdp_pc"].apply(classify_income)
# apply takes each value in gdp_pc and parses it to the function
# But this is basically a for loop under the hood.

co2[["country", "year", "gdp_pc", "income_group"]].head()
```

Out[ ]:

	country	year	gdp_pc	income_group
0	Afghanistan	2000	174.93	low
1	Afghanistan	2001	138.71	low
2	Afghanistan	2002	178.95	low
3	Afghanistan	2003	198.87	low
4	Afghanistan	2004	221.76	low

That works, and it reads well. `.apply()` takes a function and calls it once for every value in the column, collecting the answers into a new Series.

It is also, quietly, **a `for` loop**. `.apply()` does not do anything a loop could not; it just says it in one line. Your function is ordinary Python and it runs 6 240 separate times.

The vectorized way

For two possible answers, `np.where` takes a condition and the two values to choose between.

In [ ]:
```
co2["era"] = np.where(co2["year"] < 2012, "early", "late") # where is used f

print(co2["era"].value_counts())
```

```
era
early    3120
late     3120
Name: count, dtype: int64
```

For more than two, `np.select` takes a **list of conditions** and a **list of choices**, plus a `default` for anything that matches none of them.

In [107…
```
conditions = [
    co2["gdp_pc"] < 1_000,
    co2["gdp_pc"] < 10_000,
    co2["gdp_pc"] < 50_000,
]

choices = ["low", "lower-middle", "upper-middle"]

e_group"] = np.select(conditions, choices, default="high")
```

Loading [MathJax]/extensions/Safe.js

```
co2[["country", "year", "gdp_pc", "income_group"]].head()
```

Out[107…

	country	year	gdp_pc	income_group
0	Afghanistan	2000	174.93	low
1	Afghanistan	2001	138.71	low
2	Afghanistan	2002	178.95	low
3	Afghanistan	2003	198.87	low
4	Afghanistan	2004	221.76	low

Look at what `np.select` is: **the `elif` ladder from Part 1.** The conditions are tried in order and the first match wins, `default` is the final `else`, and the order of the branches is part of the logic in exactly the same way. Put `< 50_000` first and every poor country in the dataset becomes upper-middle income.

Does it matter which one you use?

Let us measure rather than assert. `time.perf_counter()` reads a clock, so the difference between two readings is how long the thing between them took.

In [119…

```
import time

start = time.perf_counter()
by_apply = co2["gdp_pc"].apply(classify_income)
apply_seconds = time.perf_counter() - start

start = time.perf_counter()
by_select = np.select(conditions, choices, default="high")
select_seconds = time.perf_counter() - start

print(f"apply:     {apply_seconds * 1000:6.2f} ms")
print(f"np.select: {select_seconds * 1000:6.2f} ms")
print(f"ratio:     {apply_seconds / select_seconds:6.1f}x")
```

```
apply:       1.74 ms
np.select:   0.32 ms
ratio:        5.5x
```

Only a few milliseconds apart. That is the honest reading of *this* measurement, and it is also why the measurement understates the case: 6 240 rows is a small table.

The gap grows with the data because `.apply()` is doing in Python what `np.select` hands to compiled code. The datasets in this course are small enough that you will rarely feel it. The datasets you meet afterwards will not be, and then efficiency will matter.

The vectorized version is also **shorter**: no function to define, no name to invent, no keep in step with the code.

Loading [MathJax]/extensions/Safe.js

> 💡 **Tip:** Use `np.where` when there are two answers and `np.select` when there are several.

Use `.apply()` only for logic that genuinely cannot be written as conditions on whole columns, which is rarely the case.

And now the part that should worry you

The two versions agree on every single row. Both of them are also **wrong**, in the same place.

In [120…
```
print("agree on every row:", (by_apply == pd.Series(by_select)).all())
```

```
agree on every row: True
```

In [121…
```
co2.loc[co2["gdp_pc"].isna(), ["country", "year", "gdp_pc", "income_group"]]
```

Out[121…

	country	year	gdp_pc	income_group
120	American Samoa	2000	NaN	high
121	American Samoa	2001	NaN	high
143	American Samoa	2023	NaN	high
720	British Virgin Islands	2000	NaN	high
721	British Virgin Islands	2001	NaN	high

In [122…
```
co2["gdp_pc"].isna().sum()
```

Out[122…
```
np.int64(166)
```

166 rows have no GDP figure at all, and both approaches have confidently labeled them **high income**.

Nothing went wrong mechanically. `NaN < 1_000` is `False`, and so is every other condition - a comparison with a missing value is never true. All three failed, and the `default` collected them. No error, no warning, and a column that looks complete.

The fix is to say what should happen to them, either before or after the comparison.

In [ ]:
```
co2["income_group"] = np.select(conditions, choices, default="high")
co2.loc[co2["gdp_pc"].isna(), "income_group"] = np.nan

print(co2["income_group"].value_counts(dropna=False))

# Alternatively (probs better) is to dropna before this.
```

Loading [MathJax]/extensions/Safe.js

```
income_group
lower-middle    2956
upper-middle    1713
low              974
high             431
NaN              166
Name: count, dtype: int64
```

> ⚠ **Warning:** A comparison with a missing value is always `False`, so missing rows fall

through every condition you write and land in whatever you put last. Before writing conditions on a column, check whether it has holes and decide what they should become.

Your turn

Starting from:

```
co2 = pd.read_csv("../data/co2_emissions.csv")
```

Add a column `renew_level` that labels each row by its renewable share: `"low"` below 20, `"medium"` below 50, and `"high"` at 50 or above. Use `np.select`, and make sure that rows with no `renew_energy` figure end up missing rather than `"high"`. Display the counts of each label, including the missing ones.

In [ ]:
```
conditions = [
    co2["renew_energy"] < 20,
    co2["renew_energy"] < 50,
]

choices = ["low", "medium"]

co2["renew_level"] = np.select(conditions, choices, default="high")

# here we fix all NaN values into nan instead of classifying it
# as high, like the code above does.
co2.loc[co2["renew_energy"].isna(), "renew_level"] = np.nan

co2["renew_level"].value_counts(dropna=False)
```

Out[ ]:
```
renew_level
low       2793
high      1415
medium    1397
NaN        635
Name: count, dtype: int64
```

Loading [MathJax]/extensions/Safe.js

Two kinds of function

We have just spent several cells arguing against writing a function and applying it to a column. That is not an argument against functions, and it is worth being clear about which is which.

A function that takes **one value** and is called once per row	usually the wrong shape. Conditions on whole columns do the same job, shorter
A function that takes **a whole table** and returns a whole table	exactly right, and it is how every analysis worth repeating is built

Everything in this notebook has been a step in one job: turn a file on disk into a table you can trust. That job is the second kind - placing it inside a function means that next time is just a single line, and the decisions inside it are written down in one place instead of scattered through a notebook.

In [ ]:
```python
def load_emissions(path):
    """
    Read the emissions file and return it ready to use.

    Parameters
    ----------
    path : str
        Path to the emissions CSV file.

    Returns
    -------
    DataFrame
        One row per country per year, with rows missing total emissions
        dropped and emissions per person added as a column.
    """
    emissions = pd.read_csv(path)

    emissions = emissions.dropna(subset=["co2_total"])
    emissions["co2_pc"] = emissions["co2_total"] * 1_000_000 / emissions["pc

    return emissions
```

In [ ]:
```python
emissions = load_emissions("../data/co2_emissions.csv")

print(emissions.shape)
emissions.head(3)
```

One line to call, a docstring saying what comes back, and every cleaning decision in one place where it can be argued with.

However, it is not finished as the table still contains 43 entities that are not countries - `World`, `Arab World`, `East Asia & Pacific` and others. Add them into a total and you count the same emissions several times over. Removing them needs a second

Loading [MathJax]/extensions/Safe.js

table saying which is which, and joining two tables together is something we do not know how to do yet.

And a first look at the shape of it

Plotting gets a session of its own later on, but one line is worth it here. We come back to how to make publication-ready figures later.

In [ ]:
```
norway = emissions[emissions["country"] == "Norway"]

norway.plot(x="year", y="co2_pc", title="Norway: tonnes of CO₂ per person")
```

A notebook is a document

From here on a notebook is not just somewhere to try things - it is a **document**. Every notebook in this half of the course, including the ones you write yourself, should:

- start by importing what it needs and loading its own data, so it does not depend on anything you ran earlier or in another notebook;
- explain in markdown what it is doing and what the output means, not only in code comments;
- run correctly from top to bottom, from a fresh start.

That last one is worth being precise about. A notebook remembers everything you have run, in the order you ran it, which is **not** the order the cells appear on screen. A notebook that works for you because of something you ran twenty minutes ago and have since deleted is a notebook that works for nobody else. Restarting and running everything from the top is how you find out.

Additional resources

- 10 minutes to pandas - the official quick tour, worth reading once you have met the ideas here
- Indexing and selecting data - the authoritative account of `[]`, `.loc` and `.iloc`, including why they differ
- NumPy: the absolute basics for beginners - arrays, dtypes and vectorization, from the source
- Working with missing data - what `NaN` is, and every tool for detecting and handling it
- SKL401 - lesson 1.10 introduces DataFrames, and lessons 3.1 to 3.6 cover importing, subsetting, filtering and missing values more briefly

Next week: combining and summarizing - answering questions about *groups* of rows rather than single ones, and joining two tables into one so that we can finally throw those

Loading [MathJax]/extensions/Safe.js

aggregates out.

Loading [MathJax]/extensions/Safe.js